1 số syntax trong lập trình socket

Tóm tắt hàm thiết lập giao tiếp, Connect, send, receive.

Khái niệm cơ bản và cú pháp

1. connect()

Dùng cho client để thiết lập kết nối đến server thông qua IP và port.

Cú pháp:

client_socket.connect((IP, PORT))

2. send()

Dùng để gửi dữ liệu giữa server và client.

Dữ liệu phải ở dạng bytes, vì socket không truyền trực tiếp kiểu string.

Ví dụ:

```
msg = "hello"
message = msg.encode("utf-8")
client_socket.send(message)
```

3. recv()

Dùng để nhận dữ liệu từ socket, dữ liệu nhận về cũng ở dạng bytes.

Sau khi nhận, cần giải mã bằng decode("utf-8") để chuyển về chuỗi gốc.

Ví dụ:

```
data = server_socket.recv(1024)   # 1024 = số bytes tối đa nhận mỗi lần
print(data.decode("utf-8"))
```

Tóm tắt syntax hàm thiết lập kết nối.

Khái niệm cơ bản:

bind(): Gán socket với địa chỉ IP và port cụ thể để server lắng nghe.

listen(): Chỉ dùng cho TCP server. Sau khi bind, server mở cổng chờ kết nối. Tham số backlog quy định số lượng kết nối đang chờ tối đa.

accept(): Khi client kết nối, hệ điều hành tạo một socket mới riêng biệt cho client đó.

Cấu trúc socket

Mô hình: 1 server – N client.

Server có socket chính để lắng nghe, mỗi client kết nối sẽ có socket riêng để trao đổi dữ liệu.

Threading

Khái niệm

Threading cho phép chương trình chạy nhiều luồng song song trong cùng một process. Trong server, threading giúp xử lý nhiều client cùng lúc thay vì tuần tự, tránh tình trạng bị block khi có nhiều request đồng thời.

Flow xử lý client trong server

-Server accept client
-Tạo thread mới cho client đó
-Thread riêng xử lý client
-Server quay lại accept client khác

Daemon Thread

- Là thread chạy nền, tự kết thúc khi chương trình chính kết thúc.
- Thường dùng cho tác vụ nền như luồng nhận tin nhắn trong client.

Ví dụ code:

```
thread = threading.Thread(target=listen, daemon=True)
thread.start()
```

Lock

-Dùng khi nhiều thread cùng truy cập tài nguyên chung.
-Đảm bảo tại một thời điểm chỉ một thread được phép thay đổi dữ liệu, tránh lỗi race condition.

Ví dụ code:

```
lock = threading.Lock()
with lock:
    clients.append(client)
```

Thread model của project

- **Server:** main thread → accept client → spawn thread → handle_client()
- **Client:** main thread (GUI) + thread nhận tin nhắn + thread gửi tin nhắn

Chạy socket trên thread riêng không block main thread

Để có thể cho server nhận được cùng lúc nhiều client thì chúng ta phải cho mỗi client chạy trên 1 thread riêng biệt, lúc này hoạt động của client sẽ không gây ảnh hưởng đến luồng hoạt động chính của server dưới đây là cách thực hiện.

Ví dụ: ta có 1 hàm **def handle_client():**, hàm này sẽ được sử dụng để nhận dữ liệu từ client, từ đó decode và in ra thông tin mà user muốn gửi.

Khi **server_socket.listen()** và đã kết nối thành công nhận được object socket của client cũng như nhận thông tin như ip và port của client sau đó ta có thể tạo 1 luồng thread riêng cho từng server như đoạn code dưới đây.

```
while True:
    # Vòng lặp vô hạn, server sẽ luôn chạy để chấp nhận nhiều kết nối từ client

    conn, addr = server.accept()
    # Chờ một client kết nối đến.
    # Khi có kết nối, trả về:
    # conn: socket riêng để giao tiếp với client
    #  addr: địa chỉ (IP, port) của client

    thread = threading.Thread(target=handle_client, args=(conn, addr))
    # Tạo một luồng (thread) mới để xử lý client vừa kết nối.
    # Hàm handle_client sẽ chạy trong luồng này, nhận tham số conn và addr.

    thread.start()
    # Bắt đầu chạy luồng mới.
    # Nhờ vậy server có thể tiếp tục lắng nghe các kết nối khác song song,
    # không bị chặn bởi một client duy nhất.
```

Kết nối Network thread và GUI thread bằng queue

trong các ứng dụng có giao diện đồ họa (GUI), dữ liệu mạng thường được xử lý ở network thread để tránh làm treo giao diện, tuy nhiên, chỉ GUI thread mới có quyền cập nhật trực tiếp giao diện, vì vậy cần một cơ chế an toàn để truyền dữ liệu giữa hai luồng.

Queue là giải pháp phổ biến vì đảm bảo dữ liệu được truyền tuần tự, tránh xung đột và dễ quản lý.

Mô hình hoạt động:

- **Network Thread:** lắng nghe dữ liệu từ socket/server và đưa dữ liệu vào queue.
- **GUI Thread:** định kỳ kiểm tra queue, lấy dữ liệu ra và cập nhật giao diện.

Sơ đồ:

[Network Thread] → [Queue] → [GUI Thread]

Giao thức & định dạng truyền tin

TCP Protocol: đảm bảo kết nối ổn định qua Internet hoặc LAN.

Protocol chung (shared/protocol.py): định nghĩa cấu trúc gói tin gồm các trường:

-**Trường cơ bản**: loại gói tin (type), người gửi (sender), người nhận (receiver – chỉ áp dụng với loại gói tin private), nội dung (content), thời gian (timestamp).

-**Trường bổ sung**:

-username: dùng cho đăng nhập, join, leave.

-message: thông báo thành công hoặc mô tả lỗi.

-reason: lý do đăng nhập thất bại.

-users: danh sách người dùng online.

Các loại gói tin:

LOGIN / LOGIN_SUCCESS / LOGIN_FAIL: dùng để đăng nhập. Client gửi username, server trả về kết quả thành công hoặc thất bại kèm lý do.

CHAT: tin nhắn công khai, gồm người gửi, nội dung, thời gian.

PRIVATE: tin nhắn riêng, gồm người gửi, người nhận, nội dung, thời gian.

JOIN / LEAVE: thông báo khi một người dùng tham gia hoặc rời phòng chat.

USER_LIST: danh sách tất cả người dùng đang online.

SYSTEM: thông báo hệ thống (ví dụ: server tắt, lỗi, thông báo chung).

ERROR: báo lỗi, kèm nội dung mô tả lỗi.

Format:

LOGIN = {
"type": "LOGIN",
"username": "",
"timestamp": ""
}
	LOGIN_SUCCESS = {
	"type": "LOGIN_SUCCESS",
	"username": "",
	"message": "",
	"timestamp": ""
	}

LOGIN_FAIL = {
"type": "LOGIN_FAIL",
"username": "",
"reason": "",
"timestamp": ""
}
	CHAT = {
	"type": "CHAT",
	"sender": "",
	"content": "",
	"timestamp": ""
	}

PRIVATE = {
"type": "PRIVATE",
"sender": "",
"receiver": "",
"content": "",
"timestamp": ""
}
	JOIN = {
	"type": "JOIN",
	"username": "",
	"timestamp": ""
	}

LEAVE = {
"type": "LEAVE",
"username": "",
"timestamp": ""
}
	USER_LIST = {
	"type": "USER_LIST",
	"users": [],
	"timestamp": ""
	}

SYSTEM = {
"type": "SYSTEM",
"content": "",
"timestamp": ""
}
	ERROR = {
	"type": "ERROR",
	"message": "",
	"timestamp": ""
	}

Áp dụng SQLite để lưu trữ tin nhắn

Server được bổ sung tính năng lưu lịch sử chat bằng SQLite. Khi user mới kết nối, server sẽ gửi lại 50 tin nhắn gần nhất để đọc lại lịch sử.

Cấu trúc file:

Code/server/

- server.py: server chính, xử lý kết nối
- database.py: module quản lý SQLite (mới)

Trong database.py có bảng messages:

```
CREATE TABLE IF NOT EXISTS messages (
id INTEGER PRIMARY KEY AUTOINCREMENT,
sender TEXT NOT NULL,
content TEXT NOT NULL,
timestamp TEXT NOT NULL,
receiver TEXT DEFAULT NULL
)
```

receiver = NULL → tin nhắn public
receiver = "username" → tin nhắn private

Các hàm chính:

- **init_db():** tạo bảng khi server khởi động
- **save_message(sender, content, timestamp, receiver=None):** lưu tin nhắn
- **get_recent_messages(limit=50):** lấy N tin nhắn public gần nhất

Thread safety: dùng threading.Lock để đảm bảo chỉ một thread ghi vào database tại một thời điểm.

Tích hợp vào server.py:

1. **Khởi tạo database khi server start:**
   database.init_db()
2. **Lưu tin nhắn khi nhận CHAT:**
   database.save_message(sender, content, timestamp)
   broadcast(msg, exclude_addr=addr)
3. **Lưu tin nhắn PRIVATE:**
   database.save_message(sender, content, timestamp, receiver)
   send_private(msg, receiver, conn)
4. **Gửi lịch sử chat khi user JOIN:**
   history = database.get_recent_messages(50)
   for h in history:
   protocol.send_message(conn, protocol.chat(h["sender"], h["content"]))

Flow tổng thể:
Server start → init_db() → Client A connect → JOIN → Server gửi user list + 50 tin nhắn cũ → Client A gửi CHAT → Server lưu tin nhắn và broadcast → Client B nhận → Client B gửi PRIVATE → Server lưu tin nhắn với receiver → chỉ Client A nhận.
Lưu ý:

- Chỉ gửi lại lịch sử public, không gửi private để bảo mật
- File database: chat_history.db tạo tại thư mục chạy server
- Với 50 tin nhắn không ảnh hưởng hiệu năng, muốn nhiều hơn thì dùng phân trang.

Tóm tắt tính năng chính của project

1. Phía Client

- Giao diện người dùng (GUI): xây dựng bằng thư viện Custom Tkinter.
- TCP connection: socket client kết nối với socket server thông qua giao thức truyền dữ liệu dùng quy tắc TCP.
- Đa luồng:
    - Luồng gửi tin nhắn (Send Thread).
    - Luồng nhận tin nhắn (Receive Thread).
    - Luồng xử lý cập nhật giao diện (Networking Thread).
- Chức năng chính: đăng nhập, gửi/nhận tin nhắn (public, private), thoát ứng dụng.

2. Phía Server

- Listen Socket: lắng nghe và chấp nhận kết nối mới từ client.
- Đa luồng: mỗi client được quản lý bởi một luồng riêng.
- Message Broker: trung tâm điều phối, nhận tin nhắn từ client và phát (broadcast) đến tất cả client (message public) và broadcast đến client cụ thể (message private).
- Quản lý dữ liệu:
    - Danh sách người dùng online.
    - Lịch sử chat (lưu trong SQLite), khi có người dùng mới đăng nhập server sẽ gửi cho client những tin nhắn đã gửi trước đó từ sql để cập nhật cho người dùng vừa vào phòng chat.
    - Lưu tin nhắn, khi có người dùng gửi tin nhắn thì client sẽ gửi tin nhắn đến server và server sẽ broadcast tin nhắn đó đến các client cũng như lưu trữ tin nhắn vào SQL để cập nhật cho các client vào sau.

4. Các thao tác chính mà người dùng có thể thực hiện thông qua GUI:

- Connect: thiết lập kết nối TCP.
- Login: xác nhận và cập nhật danh sách online.

- Chat: gửi tin nhắn, server broadcast cho các client.
- Disconnect: thoát và giải phóng tài nguyên.

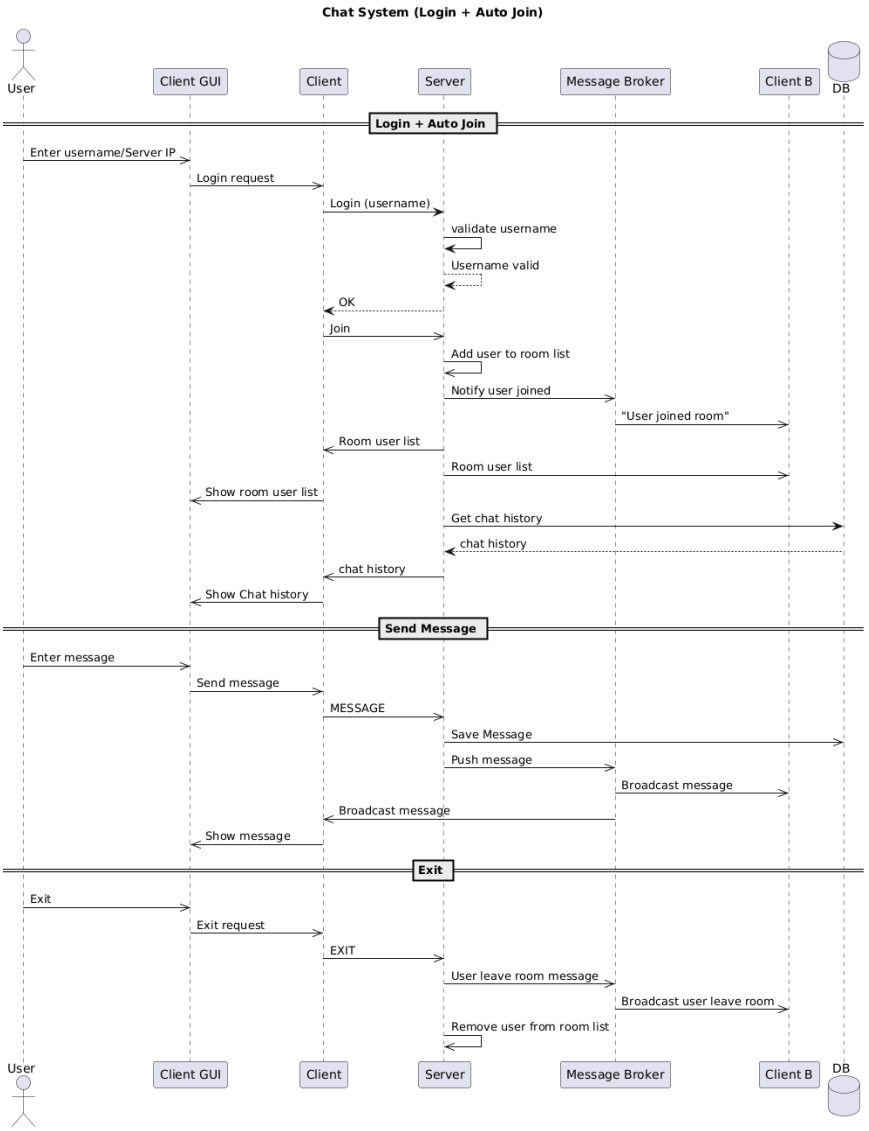
Luồng hoạt động của project

Server:

1. **socket()** → tạo socket
2. **bind(IP, PORT)** → gán địa chỉ
3. **listen** → chờ client kết nối
4. Vòng lặp:
   -accept() → nhận client
   -recv() → nhận dữ liệu
   -send() → gửi dữ liệu

Client:

1. **socket()** → tạo socket
2. **connect(server_ip, port)** → kết nối đến server
3. **send()** → gửi dữ liệu
4. **recv()** → nhận dữ liệu



Cấu trúc thư mục

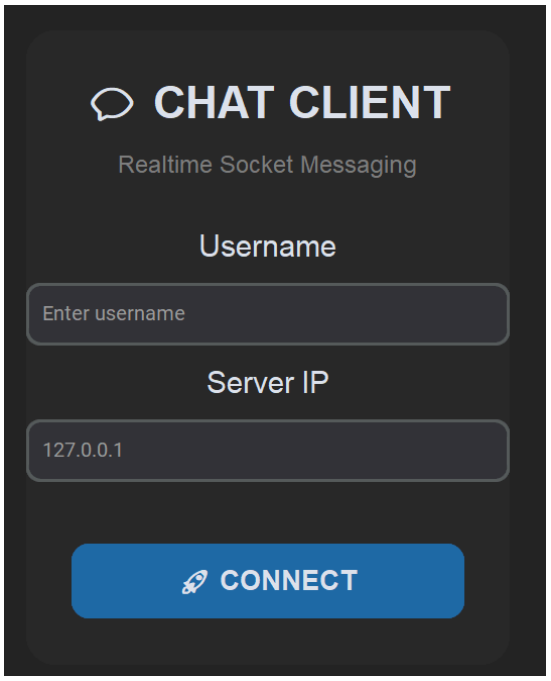
```
tcp-chat-app/
│   .gitignore
│   README.md
│   requirements.txt
│
├── Code/
│   ├── client/       # Code phía client (GUI, xử lý socket)
│   ├── server/       # Code phía server (socket, database)
│   ├── shared/       # Protocol chung (định nghĩa gói tin)
│   └── tests/        # Unit test cho protocol
│
├── DOCX/             # Tài liệu, hình ảnh mô tả
│   ├── reseach/      # Các file nghiên cứu, giải thích
```

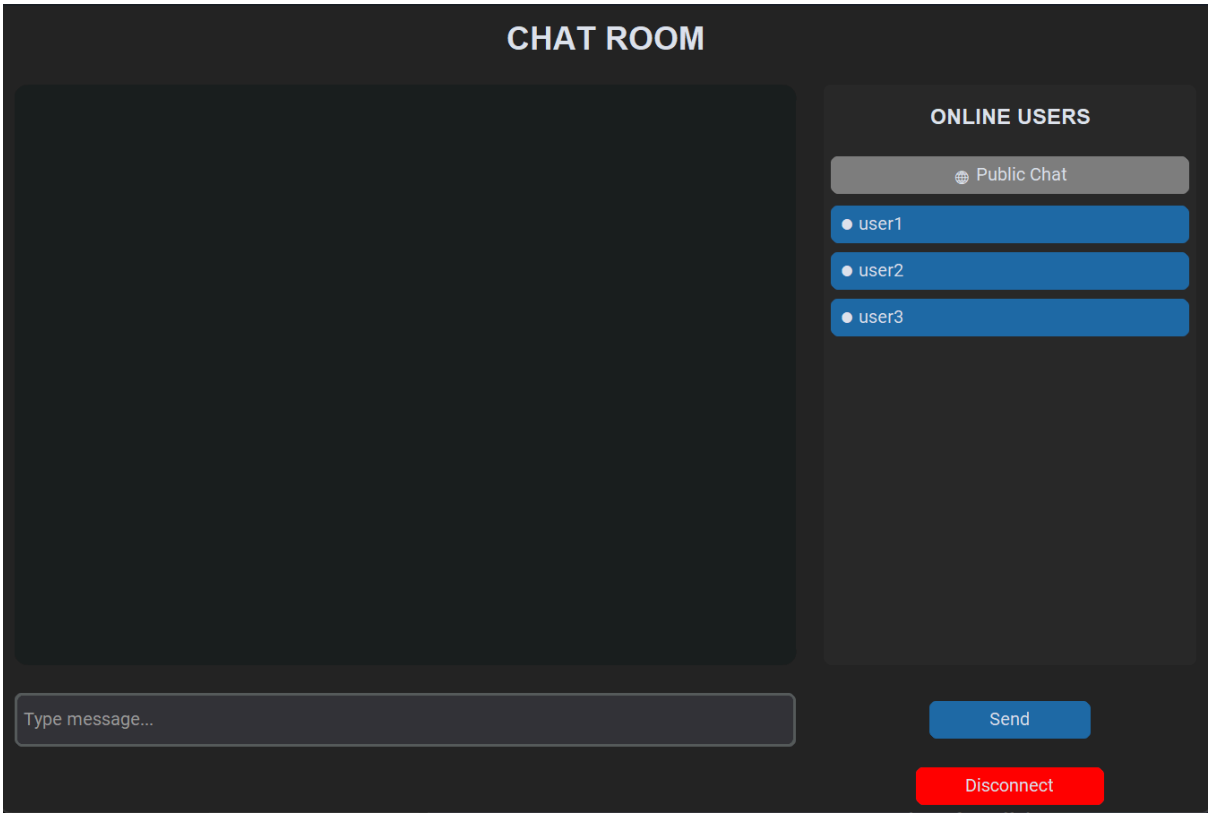
```
│   └── wireframe/     # Hình ảnh wireframe giao diện
│
├── Extra/             # File kết quả test
└── PPTX/              # Slide thuyết trình
```

Ngôn ngữ sử dụng

Python, Custom Tkinter, SQLite

Tài liệu hình ảnh




CHAT ROOM
ONLINE USERS
Public Chat
user1
user2
user3
Type message...
Send
Disconnect